

First Order Theory of Combinators

Future Work

Andrés Sicard-Ramírez

EAFIT University

(Last modification: August 31, 2012)

Future work: Theoretical (1)

- Polymorphism

Lists of (possibly partial) elements

```
data List : D → Set where
  nilL   : List []
  consL  : ∀ x {xs} → List xs → List (x :: xs)
```

Lists of total natural numbers

```
data ListN : D → Set where
  nilLN   : ListN []
  consLN  : ∀ {n ns} → N n → ListN ns → ListN (n :: ns)
```

Future work: Theoretical (1)

- Polymorphism

Lists of (possibly partial) elements

```
data List : D → Set where
  nilL   : List []
  consL  : ∀ x {xs} → List xs → List (x :: xs)
```

Lists of total natural numbers

```
data ListN : D → Set where
  nilLN   : ListN []
  consLN  : ∀ {n ns} → N n → ListN ns → ListN (n :: ns)
```

- Haskell type classes

```
class Eq a where
  (==), (/=) :: a -> a -> Bool
```

```
x /= y = not (x == y)
```

```
x == y = not (x /= y)
```

Future work: Theoretical (2)

- Relation with other programming logics (Scott, Feferman, Beeson, etc.)

Future work: Theoretical (2)

- Relation with other programming logics (Scott, Feferman, Beeson, etc.)
- Bob Harper in its blog [Haskell Is Exceptionally Unsafe](#) pointed out the equational theory with $\text{seq} : a \rightarrow b \rightarrow b$ is different to the equational theory without it.

Future work: Integration

- ① Integration with Satisfiability Modulo Theories (SMT) solvers:
 - [Alt-Ergo](#) (linear arithmetic)
 - [Z3](#) (linear real and integer arithmetic, fixed-size bit-vectors, etc)
 - [Yices](#) (linear real and integer arithmetic, recursive datatypes, lambda expressions. etc.)
 - ...

Future work: Integration

- ① Integration with Satisfiability Modulo Theories (SMT) solvers:
 - [Alt-Ergo](#) (linear arithmetic)
 - [Z3](#) (linear real and integer arithmetic, fixed-size bit-vectors, etc)
 - [Yices](#) (linear real and integer arithmetic, recursive datatypes, lambda expressions. etc.)
 - ...
- ② Integration with automatic inductive theorem provers:
 - [IsaPlanner](#)
 - [ACL2](#)
 - ...

Future work: Integration

- ① Integration with Satisfiability Modulo Theories (SMT) solvers:
 - [Alt-Ergo](#) (linear arithmetic)
 - [Z3](#) (linear real and integer arithmetic, fixed-size bit-vectors, etc)
 - [Yices](#) (linear real and integer arithmetic, recursive datatypes, lambda expressions. etc.)
 - ...
- ② Integration with automatic inductive theorem provers:
 - [IsaPlanner](#)
 - [ACL2](#)
 - ...
- ③ To merge FOTC-style for program verification with the dependently typed programming style (normalization and automatic type-checking)

Future work: Implementation

- 1 Proof reconstruction for the automatically proved theorems

Future work: Implementation

- ① Proof reconstruction for the automatically proved theorems
- ② A translator between Haskell programs and FOTC

Future work: Implementation

- ① Proof reconstruction for the automatically proved theorems
- ② A translator between Haskell programs and FOTC
- ③ Using Isabelle as a logical framework for FOTC

Future work: Others

- ① Using Agda combinators for co-induction

Future work: Others

- ① Using Agda combinators for co-induction
- ② More examples (Haskell libraries, Okasaki's book, etc.)